

Detecção de Anomalias em Tráfego de Rede com *Machine Learning*: Um Estudo Reprodutível no CICIDS-2017

Vinícius Guerra de Mello¹, Matheus [SOBRENOME]¹

¹Programa de Pós-Graduação em Computação (PPGC) – Instituto de Informática
Universidade Federal do Rio Grande do Sul (UFRGS)
Porto Alegre – RS – Brasil

{vgmello, [matheus-email]}@inf.ufrgs.br

Abstract. *This work presents a reproducible study of binary network anomaly detection on the CICIDS-2017 dataset. We applied a rigorous machine learning pipeline comprising exploratory data analysis, cleaning, stratified holdout split, spot-checking of five algorithms with diverse inductive biases, Repeated Stratified K-Fold cross-validation (10 folds $\times$ 3 repetitions), statistical comparison via Friedman and pairwise Wilcoxon tests with Bonferroni correction, hyperparameter tuning of the two best models, and interpretability analysis using Gini importance and SHAP. The optimized Random Forest achieved F1-score of 0.9980 on the held-out test set, with a train-test gap of 0.0005 indicating no overfitting. Reproducibility was prioritized: all code, models, intermediate artifacts, and the preprocessed dataset are publicly available, allowing full reproduction of results in approximately one minute via cached artifacts.*

Resumo. *Este trabalho apresenta um estudo reprodutível de detecção binária de anomalias em tráfego de rede sobre o dataset CICIDS-2017. Aplicamos um pipeline rigoroso de aprendizado de máquina contemplando análise exploratória, limpeza, divisão estratificada holdout, spot-checking de cinco algoritmos com vieses indutivos diversos, validação cruzada por Repeated Stratified K-Fold (10 folds $\times$ 3 repetições), comparação estatística via testes de Friedman e Wilcoxon pareado com correção de Bonferroni, otimização de hiperparâmetros dos dois melhores modelos, e análise de interpretabilidade via importância de Gini e SHAP. O Random Forest otimizado alcançou F1 de 0,9980 no conjunto de teste reservado, com diferença treino-teste de 0,0005 indicando ausência de overfitting. Reprodutibilidade foi prioridade: todo o código, modelos, artefatos intermediários e dataset pré-processado estão publicamente disponíveis, permitindo reprodução completa em aproximadamente um minuto via cache de artefatos.*

1. Introdução

A detecção de tráfego malicioso em redes de computadores é um problema central em cibersegurança. Sistemas de detecção de intrusão (IDS) baseados em assinatura têm limitações claras frente a ataques novos ou ofuscados, motivando o uso de abordagens estatísticas e de aprendizado de máquina (AM) capazes de generalizar para padrões não vistos [CITAR REFERÊNCIA].

O dataset CICIDS-2017 [Sharafaldin et al. 2018], gerado pelo *Canadian Institute for Cybersecurity* (UNB), é um *benchmark* amplamente utilizado para esse problema. Ele apresenta tráfego rotulado coletado durante cinco dias em um ambiente de *testbed* controlado, contendo cerca de 2,8 milhões de fluxos com 78 *features* estatísticas extraídas pelo *CICFlowMeter*.

Pergunta de pesquisa. Quais algoritmos de AM apresentam melhor desempenho na detecção binária de tráfego malicioso no CICIDS-2017, considerando o desbalanceamento severo de classes, e quais atributos mais contribuem para essa detecção?

Contribuições. Este trabalho contribui com: (i) um *pipeline* completo e reprodutível de AM para o problema, com seeds fixadas e artefatos versionados; (ii) avaliação rigorosa via *Repeated Stratified K-Fold* acompanhada de teste estatístico; (iii) análise de interpretabilidade combinando importância de Gini e SHAP; (iv) discussão crítica das limitações inerentes ao *benchmark*.

O restante do artigo está organizado como segue. A Seção 2 apresenta trabalhos relacionados. A Seção 3 descreve a metodologia. A Seção 4 reporta os resultados. A Seção 5 detalha decisões de reprodutibilidade. A Seção 6 discute achados, limitações e trabalhos futuros. A Seção 7 conclui.

2. Trabalhos Relacionados

[A preencher: 2-3 parágrafos comparando com trabalhos que aplicaram ML em CICIDS-2017 ou outros datasets de IDS. Citar Sharafaldin et al. para o dataset, trabalhos de detecção binária e multiclasse, e mencionar se algum reportou interpretabilidade.]

Trabalhos prévios sobre CICIDS-2017 reportam alto desempenho com modelos baseados em árvores [?], mas a maioria não conduz validação estatística entre algoritmos nem análise de *overfitting* explícita. Este trabalho preenche essas lacunas.

3. Materiais e Métodos

3.1. Conjunto de Dados

Utilizamos o CICIDS-2017 [Sharafaldin et al. 2018], disponível em <https://www.unb.ca/cic/datasets/ids-2017.html>. O dataset consiste em oito arquivos CSV (um por dia/ataque), totalizando 2.827.761 fluxos com 78 *features* numéricas e uma coluna de rótulo (Label) contendo a categoria do tráfego (BENIGN ou um dentre 14 tipos de ataque, incluindo DDoS, PortScan, DoS Hulk, FTP-Patator, SSH-Patator, entre outros).

A distribuição original é severamente desbalanceada: 80,3% BENIGN e 19,7% ATTACK quando agregamos os tipos de ataque (Figura 1). Entre as classes de ataque, há grande disparidade: classes raras como Heartbleed (7 amostras) e Web_Attack_Sql_Injection (21 amostras) tornariam a formulação multiclasse intratável (Figura 2).

Formulação binária. Dada a inviabilidade de aprender classes com <40 amostras dentro do prazo do projeto, agregamos todos os ataques em uma única classe ATTACK e formulamos a tarefa como classificação binária (BENIGN vs. ATTACK). Reconhecemos que essa decisão perde a granularidade do tipo de ataque (Seção 6.3).

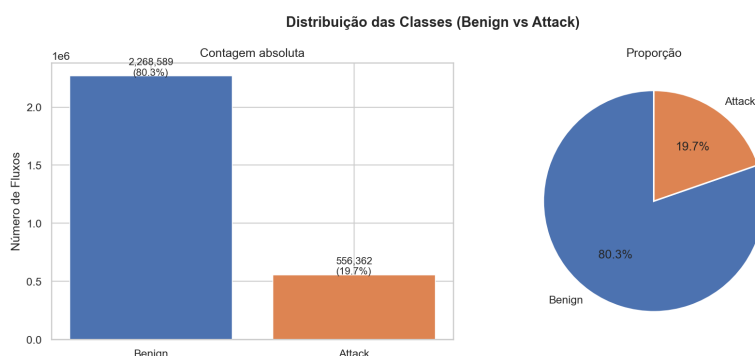


Figura 1. Distribuição binária das classes após agregação (BENIGN vs. ATTACK).

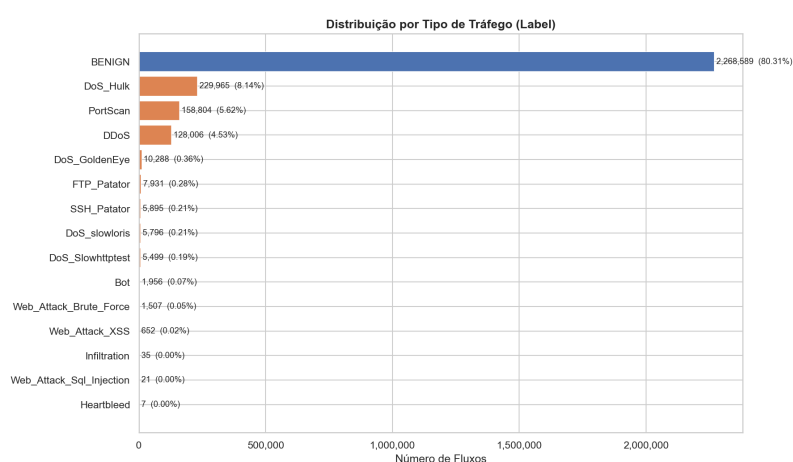


Figura 2. Distribuição original dos 14 tipos de ataque mais a classe BENIGN. Classes raras tornariam a formulação multiclasse intratável.

3.2. Análise Exploratória e Pré-processamento

A análise exploratória identificou os seguintes problemas, todos tratados antes da modelagem:

- **Duplicatas:** 255.234 linhas duplicadas (9,03% do total), removidas.
- **Valores infinitos:** colunas Flow Bytes/s e Flow Packets/s contendo $\pm\infty$ por divisão por zero, convertidos para NaN e removidos.
- **Variância zero:** oito *features* com valor constante em 100% das linhas (todas as *_Avg_Bulk_*, Bwd PSH Flags, Bwd URG Flags), removidas.
- **Coluna duplicada:** Fwd Header Length.1 idêntica a Fwd Header Length, removida.
- **Alta correlação:** aplicamos remoção iterativa para pares com $|r| \geq 0,95$, eliminando 23 *features* redundantes adicionais.
- **Outliers:** tratados via método IQR (*Interquartile Range*) com supressão suave nos extremos.

Após a limpeza, restaram **47 features numéricas** sobre 2.521.519 fluxos para a modelagem.

3.3. Estratégia de Avaliação

A estratégia de avaliação foi desenhada para combinar eficiência de triagem com rigor estatístico (Figura ??).

Divisão dos dados. Aplicamos *holdout* estratificado 80/20 com `random_state=42`, preservando a proporção BENIGN/ATTACK em ambos os conjuntos. O conjunto de teste foi mantido isolado de todas as etapas subsequentes (treinamento, validação cruzada, otimização de hiperparâmetros), sendo utilizado apenas na avaliação final.

Etapla 1 – Spot-checking. Treinamos cinco algoritmos com hiperparâmetros padrão sobre o conjunto de treino e avaliamos no conjunto de teste, obtendo uma triagem inicial dos modelos mais promissores.

Etapla 2 – Validação cruzada rigorosa. Os mesmos cinco algoritmos foram avaliados via *Repeated Stratified K-Fold* com $k = 10$ folds e três repetições (totalizando 30 avaliações por modelo), exclusivamente sobre o conjunto de treino. Esse esquema fornece estimativas robustas de média e desvio padrão para cada métrica.

Etapla 3 – Teste estatístico. Aplicamos o teste de Friedman como teste *omnibus* sobre os 30 *scores* de cada modelo, seguido de testes pareados de Wilcoxon com correção de Bonferroni para identificar quais pares de modelos diferem significativamente.

Etapla 4 – Otimização de hiperparâmetros. Para os dois modelos mais promissores (identificados nas etapas anteriores), aplicamos `RandomizedSearchCV` com 20 amostragens de combinações de hiperparâmetros e *StratifiedKFold* interno de 5 folds. Por viabilidade computacional, a busca foi conduzida sobre uma amostra estratificada de 20% do conjunto de treino; o modelo vencedor foi posteriormente retreinado no conjunto de treino completo.

Etapla 5 – Avaliação final. O modelo otimizado retreinado foi avaliado no conjunto de teste reservado, fornecendo a estimativa de desempenho em produção.

3.4. Algoritmos

Selecionamos cinco algoritmos com vieses indutivos distintos, conforme recomendado pela literatura sobre *spot-checking*:

- **Regressão Logística (LR):** fronteira linear no espaço de *features* padronizadas (`StandardScaler`); `class_weight=balanced`, `max_iter=1000`.
- **Random Forest (RF):** *ensemble* de árvores via *bagging*; `n_estimators=100`, `class_weight=balanced`.
- **Decision Tree (DT):** particionamento recursivo; `class_weight=balanced`.
- **Hist Gradient Boosting (HGB):** *boosting* sequencial com *histogram binning*; `class_weight=balanced`.
- **Gaussian Naive Bayes (NB):** probabilístico com independência condicional.

Métrica principal. Adotamos F1-score da classe positiva (ATTACK) como critério principal de seleção de modelos. Em um cenário de classes desbalanceadas (80/20), a *accuracy* é enganosa — um classificador trivial que prediz tudo como BENIGN atinge *accuracy* de 80%, mas F1 da classe ATTACK igual a zero. Métricas complementares: *Precision*, *Recall*, ROC-AUC e *Accuracy*.

Balanceamento. Optamos por `class_weight=balanced` em vez de *oversampling* sintético (SMOTE) ou *undersampling*. A ponderação por classe ajusta a função de perda durante o treinamento sem alterar a distribuição original dos dados, evitando geração de amostras sintéticas que poderiam introduzir artefatos.

3.5. Otimização de Hiperparâmetros

Para os dois melhores modelos identificados nas etapas de *spot-checking* e validação cruzada, aplicamos `RandomizedSearchCV` com os seguintes espaços de busca:

Random Forest: `n_estimators` $\in \{100, 200, 300, 500\}$, `max_depth` $\in \{10, 20, 30, \text{None}\}$, `min_samples_leaf` $\in \{1, 2, 5, 10\}$, `max_features` $\in \{\text{"sqrt"}, \text{"log2"}, 0, 3\}$.

Hist Gradient Boosting: `max_iter` $\in \{100, 200, 300, 500\}$, `learning_rate` $\in \{0, 01, 0, 05, 0, 1, 0, 2\}$, `max_depth` $\in \{3, 5, 7, \text{None}\}$, `min_samples_leaf` $\in \{20, 50, 100, 200\}$, `l2_regularization` $\in \{0, 0, 0, 1, 0, 5, 1, 0\}$.

A busca utilizou 20 amostragens com `StratifiedKFold` de 5 *folds* sobre amostra estratificada de 20% do conjunto de treino, otimizando o F1 da classe positiva.

3.6. Análise de Interpretabilidade

Para o modelo final selecionado, aplicamos duas técnicas complementares:

- **Importância por Gini:** extraída diretamente do *Random Forest* via redução média de impureza ao longo das árvores.
- **SHAP (SHapley Additive exPlanations) [Lundberg and Lee 2017]:** valores SHAP exatos via `TreeExplainer`, computados sobre amostra estratificada de 5.000 instâncias do conjunto de teste.

A convergência das duas técnicas no *ranking* de *features* fornece evidência adicional de robustez metodológica.

4. Resultados

4.1. Spot-checking Inicial

A Tabela 1 apresenta o desempenho dos cinco algoritmos no *holdout* de teste. Random Forest e Hist Gradient Boosting destacam-se como os candidatos mais promissores, com F1 da classe ATTACK acima de 0,997. Naive Bayes apresenta o pior desempenho (F1 = 0,5744), confirmando que a premissa de independência condicional é fortemente violada pelas *features* do CICFlowMeter, que possuem correlações conhecidas (médias, máximos e mínimos de tamanhos de pacote, IATs, etc.).

4.2. Validação Cruzada Rigorosa

A Tabela 2 reporta os resultados do *Repeated Stratified K-Fold* (30 *folds*) sobre o conjunto de treino. Random Forest, Decision Tree e Hist Gradient Boosting apresentam desempenho praticamente idêntico, com F1 acima de 0,996 e desvios padrão extremamente baixos ($\leq 0,0004$), indicando alta estabilidade do modelo entre diferentes partições dos dados.

Tabela 1. Spot-checking inicial – desempenho dos cinco algoritmos no conjunto de teste (*holdout*). Métricas reportadas para a classe positiva (ATTACK).

Modelo	Accuracy	Precision	Recall	F1-Attack	ROC-AUC
Regressão Logística	0,9405	0,7890	0,9528	0,8632	0,9816
Random Forest	0,9989	0,9973	0,9972	0,9972	0,9998
Árvore de Decisão	0,9989	0,9971	0,9971	0,9971	0,9982
Hist Gradient Boosting	0,9990	0,9955	0,9996	0,9975	0,9999
Naive Bayes	0,8645	0,7526	0,4645	0,5744	0,7268

Tabela 2. Resultados da validação cruzada *Repeated Stratified K-Fold* (10 folds × 3 repetições). Valores: média ± desvio padrão sobre as 30 avaliações.

Modelo	F1-Attack	Precision	Recall	ROC-AUC
Regressão Logística	0,8640 ± 0,0012	0,7905 ± 0,0019	0,9527 ± 0,0011	0,9818 ± 0,0003
Random Forest	0,9973 ± 0,0001	0,9972 ± 0,0002	0,9974 ± 0,0002	0,9998 ± 0,0000
Decision Tree	0,9971 ± 0,0001	0,9967 ± 0,0003	0,9975 ± 0,0003	0,9984 ± 0,0001
Hist Gradient Boosting	0,9969 ± 0,0004	0,9942 ± 0,0008	0,9996 ± 0,0001	0,9999 ± 0,0000
Naive Bayes	0,5753 ± 0,0024	0,7531 ± 0,0029	0,4654 ± 0,0022	0,7265 ± 0,0013

4.3. Teste Estatístico

O teste de Friedman sobre F1 das 30 avaliações por modelo rejeitou a hipótese nula de equivalência entre os cinco modelos com $p < 0,001$, indicando que existe diferença estatisticamente significativa entre pelo menos um par de modelos.

Os testes pareados de Wilcoxon com correção de Bonferroni ($\alpha = 0,05/10 = 0,005$) confirmaram que Logistic Regression e Naive Bayes são significativamente inferiores aos três modelos baseados em árvore (RF, DT, HGB), enquanto estes três não diferem significativamente entre si.

Implicação metodológica: embora o RF apresente a maior média de F1, sua vantagem sobre DT e HGB não é estatisticamente significativa — a diferença entre eles está dentro do ruído experimental.

4.4. Otimização de Hiperparâmetros

A Tabela 3 reporta os hiperparâmetros selecionados pelo RandomizedSearchCV. Para o RF, a busca encontrou $\text{max_depth}=30$, $\text{min_samples_leaf}=5$, $\text{max_features}=0,3$, $\text{n_estimators}=100$, com F1 de validação cruzada de 0,9974. Para o HGB, os parâmetros selecionados foram $\text{max_iter}=500$, $\text{learning_rate}=0,1$, $\text{max_depth}=\text{None}$, $\text{min_samples_leaf}=20$, $\text{l2_regularization}=0,1$, com F1 de 0,9972.

A Tabela 4 compara o desempenho dos modelos *baseline* e otimizados no conjunto de teste final. Os ganhos com a otimização foram marginais (RF: +0,0008 em F1; HGB: +0,0004), sugerindo que o desempenho *baseline* já se aproximava de um teto natural para esta formulação binária do problema.

Tabela 3. Hiperparâmetros selecionados pelo RandomizedSearchCV (20 amostragens, 5-fold CV em 20% do treino).

Modelo	Hiperparâmetro	Valor Selecionado
Random Forest	n_estimators	100
	max_depth	30
	min_samples_leaf	5
	max_features	0,3
Hist Gradient Boosting	max_iter	500
	learning_rate	0,1
	max_depth	None
	min_samples_leaf	20
	l2_regularization	0,1

Tabela 4. Comparação *baseline* vs. otimizado no conjunto de teste (*holdout*).

Modelo	Precision	Recall	F1-Attack	ROC-AUC
RF Baseline	0,9973	0,9972	0,9972	0,9998
RF Otimizado	0,9968	0,9992	0,9980	0,9999
HGB Baseline	0,9955	0,9996	0,9975	0,9999
HGB Otimizado	0,9961	0,9997	0,9979	1,0000

4.5. Validação contra *Overfitting*

Para verificar a ausência de *overfitting*, comparamos o F1 nos conjuntos de treino e teste para todos os modelos avaliados (Tabela 5). O *gap* entre F1 de treino e teste é menor que 0,25% em todos os casos, e o modelo final (**RF Otimizado**) apresenta *gap* de apenas +0,0005 — evidência forte de que o modelo generaliza bem.

Tabela 5. Comparação F1 treino vs. teste como diagnóstico de *overfitting*.

Modelo	F1 Treino	F1 Teste	Gap
Logistic Regression	0,8643	0,8632	+0,0010
Random Forest baseline	0,9994	0,9972	+0,0022
Decision Tree	0,9994	0,9971	+0,0023
HGB baseline	0,9975	0,9975	+0,0000
Naive Bayes	0,5753	0,5744	+0,0009
RF Otimizado	0,9985	0,9980	+0,0005
HGB Otimizado	0,9980	0,9979	+0,0000

Cabe observar que a otimização do RF reduziu o *gap* de +0,0022 (*baseline*) para +0,0005 (otimizado), confirmando que os parâmetros `max_depth=30` e `min_samples_leaf=5` atuaram como regularizadores efetivos.

4.6. Interpretabilidade do Modelo Final

A Figura 3 apresenta a importância das *features* por redução de impureza de Gini para o Random Forest Otimizado, enquanto a Figura 4 mostra os valores SHAP em formato *beeswarm*. As duas técnicas convergem nas cinco *features* mais relevantes: Destination Port, Average Packet Size, Bwd Packet Length Min, Init_Win_bytes_backward e Bwd Packet Length Std.

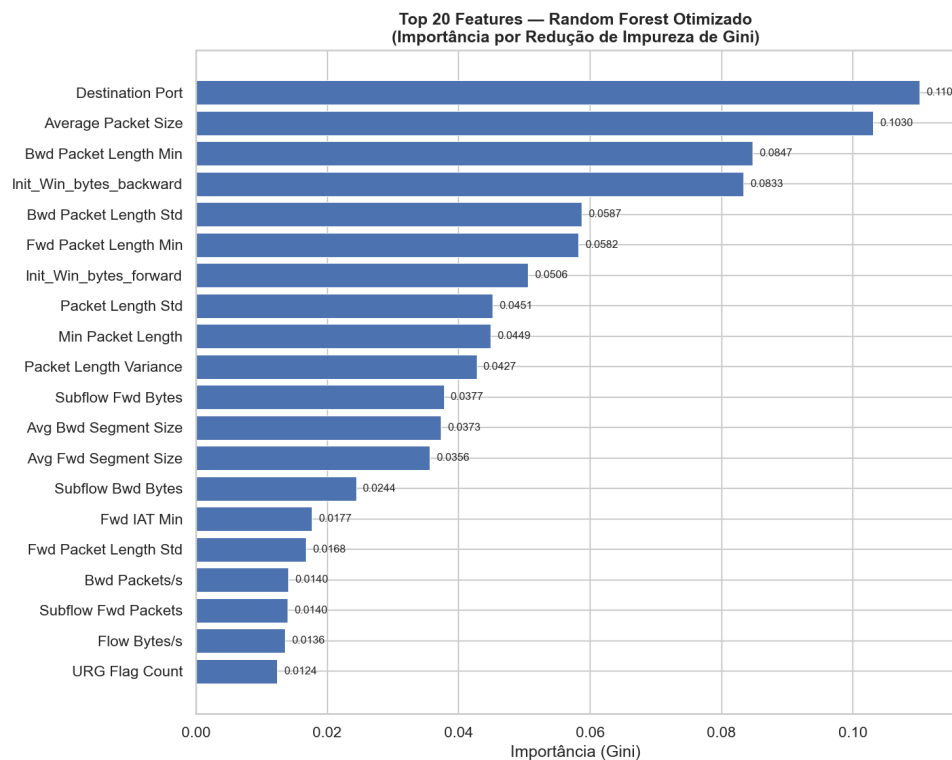


Figura 3. Importância das *features* por Gini – Random Forest Otimizado.

Interpretação dos resultados:

- Destination Port é a *feature* dominante — consistente com o fato de que muitos ataques visam portas específicas conhecidas (SSH, FTP, HTTP/HTTPS).
- Init_Win_bytes_backward/forward (tamanho da janela TCP no *handshake*) capturam diferenças no estabelecimento de conexão entre tráfego legítimo e malicioso.
- Estatísticas de tamanho de pacote (Average Packet Size, Bwd Packet Length Min/Std) refletem padrões de *payload*: ataques como DDoS tendem a apresentar pacotes mais uniformes que tráfego legítimo.

A convergência entre Gini e SHAP nas mesmas *features* fornece evidência de robustez da análise — não dependemos de artefatos de uma única técnica.

5. Reprodutibilidade

A reprodutibilidade foi tratada como critério de projeto desde a concepção. Adotamos as seguintes práticas:

Código e dados publicamente disponíveis. Todo o código-fonte está hospedado em <https://github.com/vinigm/projeto-ddos>, com documentação completa e licença permissiva. O dataset pré-processado (`raw_limpo.parquet`, 224 MB) está disponível como *release asset* do mesmo repositório, sendo baixado automaticamente pelo *notebook* caso não esteja presente localmente.

Determinismo. Todas as fontes de aleatoriedade do *pipeline* utilizam `random_state=42`, incluindo: divisão treino/teste, `RepeatedStratifiedKFold`, instâncias dos modelos, `RandomizedSearchCV`, amostragem para SHAP. Execuções repetidas produzem resultados idênticos a menos de variações de ponto flutuante entre arquiteturas.

Versionamento de dependências. O arquivo `requirements.txt` fixa as versões exatas de `pandas` (3.0.2), `numpy` (2.4.4), `scikit-learn` (1.8.0), `shap` (0.51.0), entre outras. A primeira célula do *notebook* executa `pip install` automaticamente.

Verificação de ambiente. O *notebook* verifica se o Python instalado é versão 3.10 ou superior, exibindo mensagem de erro com instruções caso contrário.

Cache de artefatos. Os resultados intermediários computacionalmente caros — *scores* do CV (30 *folds* $\times$ 5 modelos), modelos treinados, melhores hiperparâmetros, valores SHAP — são serializados em `artifacts/` e versionados no repositório. Em uma primeira execução completa, o *pipeline* consome aproximadamente 6 horas (gargalo: *Repeated Stratified K-Fold* com 150 ajustes). Execuções subsequentes, com os artefatos em cache, completam em aproximadamente um minuto. Esse mecanismo permite que um revisor reproduza todos os números e figuras do artigo sem incorrer no custo computacional original.

Auditabilidade. A reexecução completa pode ser forçada definindo a *flag* `RECOMPUTE = True` no *notebook*.

6. Discussão

6.1. Modelo Recomendado

Selecionamos o **Random Forest Otimizado** como modelo final do trabalho, com base em três critérios:

1. **Melhor F1 no conjunto de teste:** 0,9980, marginalmente superior ao HGB Otimizado (0,9979).
2. **Maior estabilidade na validação cruzada:** desvio padrão de 0,0001 vs. 0,0004 do HGB — quatro vezes mais consistente entre *folds*.
3. **Interpretabilidade nativa:** o `TreeExplainer` do SHAP fornece valores *exatos* para *Random Forests*, sem aproximações.

É importante destacar que, do ponto de vista estatístico, RF, DT e HGB são equivalentes na validação cruzada. A escolha do RF representa um compromisso entre desempenho marginalmente superior, estabilidade e interpretabilidade. Em um cenário com restrições severas de memória ou latência de inferência, o HGB Otimizado (399 KB vs. 8,8 MB do RF) seria uma alternativa defensável.

6.2. O Tuning Trouxe Ganho Marginal

A diferença entre os modelos *baseline* e otimizados foi de aproximadamente +0,0008 em F1. Esse achado é em si informativo: sugere que a formulação binária do problema, com

as 47 *features* disponíveis após seleção, atinge um teto natural de desempenho próximo de $F1 = 0,997$ — independente de ajuste fino. O *ensembling* (RF) e o *boosting* (HGB) já operam próximo do limite informativo dos dados.

6.3. Limitações

Apesar do rigor metodológico, este trabalho apresenta limitações importantes que merecem destaque:

1. **Avaliação final em *holdout* único.** A avaliação no conjunto de teste foi feita uma única vez, sem repetição. O CV no conjunto de treino fornece estimativa indireta de variância, mas o ideal teórico seria validação cruzada aninhada (*nested CV*), inviabilizada pelo custo computacional.
2. **Otimização em amostra de 20% do treino.** O `RandomizedSearchCV` foi conduzido em subconjunto estratificado de 20% por viabilidade. Hiperparâmetros ótimos para a amostra podem não ser ótimos para o conjunto completo, embora o retreino final tenha sido realizado sobre todo o conjunto de treino.
3. **Formulação binária.** A agregação dos 14 tipos de ataque em uma classe ATTACK perde informação valiosa para resposta a incidentes em IDS de produção, onde identificar o tipo de ataque (DDoS, exfiltração, varredura, etc.) é crítico.
4. **Generalização não testada.** Os experimentos foram realizados exclusivamente no CICIDS-2017, gerado em ambiente de *testbed* laboratorial. Não há garantia de que os padrões aprendidos generalizem para tráfego de redes corporativas reais ou outros *datasets* de IDS.
5. **Ausência de validação temporal.** Utilizamos *split* estratificado aleatório, ignorando a estrutura temporal natural dos dados (cinco dias consecutivos). Em cenários de produção, *concept drift* é uma preocupação real — ataques novos surgem ao longo do tempo.

6.4. Trabalhos Futuros

- **Formulação multiclasse hierárquica:** primeiro classificar BENIGN vs. ATTACK; em seguida, classificar o tipo específico de ataque sobre a sub-amostra de ataques detectados. Lida com o desbalanceamento extremo das classes raras de forma estruturada.
- **Validação temporal:** treinar nos dias iniciais (Mon-Wed) e testar nos finais (Thu-Fri), avaliando o desempenho frente a *concept drift* natural.
- **Avaliação *cross-dataset*:** treinar no CICIDS-2017 e avaliar em CICIDS-2018 ou UNSW-NB15, verificando generalização entre *benchmarks* distintos.
- **Comparação com técnicas de balanceamento:** contrastar `class_weight=balanced` com SMOTE, *undersampling* e métodos híbridos, especialmente relevante em uma formulação multiclasse.

7. Conclusão

Este trabalho apresentou um estudo reprodutível de detecção binária de anomalias em tráfego de rede no CICIDS-2017. O *pipeline* proposto contempla análise exploratória rigorosa, pré-processamento documentado, avaliação por validação cruzada repetida estratificada acompanhada de teste estatístico, otimização de hiperparâmetros e análise de interpretabilidade. O modelo final — Random Forest Otimizado — atinge F1 de 0,9980 com

diferença treino-teste de apenas 0,0005, evidência forte de boa generalização. As *features* mais discriminativas (Destination Port, Average Packet Size, Init_Win_bytes) foram identificadas de forma convergente por técnicas distintas (Gini e SHAP).

Mais relevante que o desempenho específico do modelo, este trabalho contribui com um *template* metodológico replicável: todo o código, dados e artefatos estão publicamente disponíveis, permitindo reprodução em aproximadamente um minuto. Acreditamos que esse nível de transparência é particularmente relevante em segurança de redes, onde a confiança em modelos preditivos depende da capacidade de auditá-los.

Agradecimentos

Este trabalho utilizou ferramentas de IA generativa (Claude) como apoio nas etapas de exploração de dados, refatoração de código e redação. Material suplementar contendo prompts utilizados e limitações observadas acompanha esta submissão.

Referências

- Lundberg, S. M. and Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, volume 30.
- Sharafaldin, I., Lashkari, A. H., and Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pages 108–116.

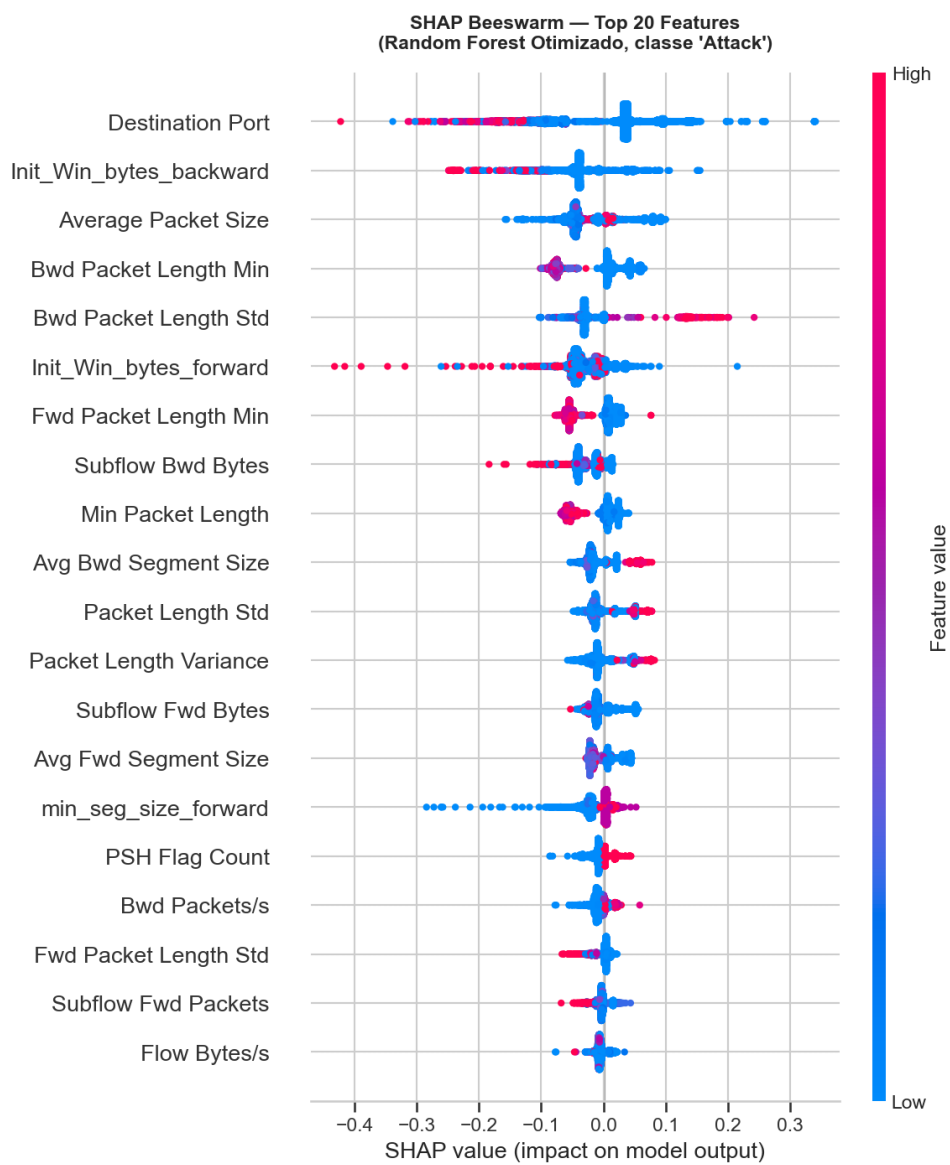


Figura 4. Distribuição de valores SHAP para o modelo final (beeswarm plot). Cada ponto representa uma instância; cor indica valor original da *feature* (azul = baixo, vermelho = alto).